

Тема: ПР 24 Применение коллизий

Цель: совершенствование навыков составления программ с библиотекой Pygame

Задачи:

- ✓ повторить структуру алгоритмов решения задач с библиотекой Pygame;
- ✓ повторить синтаксис операторов работы с графикой;
- ✓ приобрести навыки составления программ с графикой.

Теоретический материал

В программировании игр спрайтом называют объект, который предстает перед пользователем в виде анимированного изображения и в большинстве случаев предполагает взаимодействие с ним. Другими словами, все что в игре не является фоном, а интерактивным объектом-картинкой – это спрайт.

Хотя каждый спрайт может быть уникальным, у всех есть нечто общее, что в pygame вынесено в отдельный класс `Sprite`, находящийся в модуле `pygame.sprite`.

На базе этого класса следует создавать собственные классы спрайтов и уже от них объекты. Таким образом, класс `pygame.sprite.Sprite` играет роль своего рода абстрактного класса. Хотя таковым не является, можно создавать объекты непосредственно от `Sprite`.

В модуле `pygame.sprite` кроме класса `Sprite` есть класс `Group` и родственные ему, которые предназначены для объединения спрайтов в группы. Это позволяет вызывать один метод группы, который, например, обновит состояние всех спрайтов, входящих в эту группу.

Почти все предопределенные методы класса `pygame.sprite.Sprite` касаются добавления экземпляра в группу, удаления из нее, проверки вхождения. Только метод `update()` затрагивает поведение самого спрайта, этот метод следует переопределить в производном от `Sprite` классе.

Рассмотрим, как это работает. В примерах кода ниже сначала одна, а потом и множество машинок перемещаются сверху вниз. Каждая такая машинка – объект-спрайт, созданный от класса `Car`, который является дочерним от `Sprite`.

В конструкторе производного от `Sprite` класса необходимо вызвать конструктор родительского класса, а также объявить экземпляры `Surface` и `Rect`, имена которых должны быть соответственно `self.image` и `self.rect`. Так надо, чтобы с экземплярами класса могли работать методы группы. В остальном вы можете добавлять любые атрибуты.

Как создается поверхность (а также прямоугольная область), неважно. В примере ниже это делается с помощью функции `load()`. Однако в конструктор может передаваться уже подготовленный экземпляр `Surface`.

```
from random import randint
import pygame as pg
import sys

W = 400
H = 400
WHITE = (255, 255, 255)

class Car(pg.sprite.Sprite):
    def __init__(self, x, filename):
        pg.sprite.Sprite.__init__(self)
        self.image = pg.image.load(
            filename).convert_alpha()
        self.rect = self.image.get_rect(
            center=(x, 0))

sc = pg.display.set_mode((W, H))

# координата x будет случайна
car1 = Car(randint(1, W), 'car1.png')

while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()

    sc.fill(WHITE)
    sc.blit(car1.image, car1.rect)
    pg.display.update()
    pg.time.delay(20)

    # машинка ездит сверху вниз
    if car1.rect.y < H:
        car1.rect.y += 2
    else:
        car1.rect.y = 0
```

В данном случае мы изменяем свойства экземпляра за пределами класса. Правильней будет делать это в методе `update()`:

```
...
class Car(pg.sprite.Sprite):
    def __init__(self, x, filename):
        pg.sprite.Sprite.__init__(self)
        self.image = pg.image.load(
            filename).convert_alpha()
        self.rect = self.image.get_rect(
            center=(x, 0))

    def update(self):
```

```
if self.rect.y < H:
    self.rect.y += 2
else:
    self.rect.y = 0

sc = pg.display.set_mode((W, H))

# координата x будет случайна
car1 = Car(randint(1, W), 'car1.png')

while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()

    sc.fill(WHITE)
    sc.blit(car1.image, car1.rect)
    pg.display.update()
    pg.time.delay(20)

    car1.update()
```

Теперь представим, что у нас не одна машинка, а три:



```
...
car1 = Car(randint(1, W), 'car1.png')
car2 = Car(randint(1, W), 'car2.png')
car3 = Car(randint(1, W), 'car3.png')

while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()

    sc.fill(WHITE)
    sc.blit(car1.image, car1.rect)
    sc.blit(car2.image, car2.rect)
    sc.blit(car3.image, car3.rect)
    pg.display.update()
    pg.time.delay(20)

    car1.update()
    car2.update()
    car3.update()
```

Если будет 100 машинок, придется 100 раз вызывать blit() и update(). Класс Group решает эту проблему. Добавлять спрайты в группу можно методом add() группы (по одной или все вместе).

У групп есть методы `update()` и `draw()`. Метод `update()` группы вызывает методы `update()` всех входящих в нее объектов. А метод `draw()` выполняет метод `blit()`. При этом в `draw()` надо передать поверхность, на которой будет происходить отрисовка:

```
...
cars = pg.sprite.Group()
cars.add(Car(randint(1, W), 'car1.png'),
         Car(randint(1, W), 'car2.png'))
cars.add(Car(randint(1, W), 'car3.png'))

while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()

    sc.fill(WHITE)

    cars.draw(sc)

    pg.display.update()
    pg.time.delay(20)

    cars.update()
```

Допустим, мы хотим, чтобы новые машинки появлялись постоянно и в разные моменты времени, двигались с разной скоростью, а выезд объекта за пределы экрана обозначал бы, что он исчезает.

Потребуется таймер, который устанавливается вызовом функции `pygame.time.set_timer()`. В примере ниже через каждые 3 секунды будет генерироваться событие, значение поля `type` которого совпадает с константой `pygame.USEREVENT`. И как только это событие будет происходить, будет создаваться новый объект.

```
from random import randint
import pygame as pg
import sys

pg.time.set_timer(pg.USEREVENT, 3000)

W = 400
H = 400
WHITE = (255, 255, 255)
CARS = ('car1.png', 'car2.png', 'car3.png')
# для хранения готовых машин-поверхностей
CARS_SURF = []

# надо установить видео режим
# до вызова image.load()
sc = pg.display.set_mode((W, H))

for i in range(len(CARS)):
    CARS_SURF.append(
```

```
pg.image.load(CARS[i]).convert_alpha())

class Car(pg.sprite.Sprite):
    def __init__(self, x, surf, group):
        pg.sprite.Sprite.__init__(self)
        self.image = surf
        self.rect = self.image.get_rect(
            center=(x, 0))
        # добавляем в группу
        self.add(group)
        # у машин будет разная скорость
        self.speed = randint(1, 3)

    def update(self):
        if self.rect.y < H:
            self.rect.y += self.speed
        else:
            # теперь не перебрасываем вверх,
            # а удаляем из всех групп
            self.kill()

cars = pg.sprite.Group()

# добавляем первую машину,
# которая появляется сразу
Car(randint(1, W),
    CARS_SURF[randint(0, 2)], cars)

while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()
        elif i.type == pg.USEREVENT:
            Car(randint(1, W),
                CARS_SURF[randint(0, 2)], cars)

    sc.fill(WHITE)

    cars.draw(sc)

    pg.display.update()
    pg.time.delay(20)

    cars.update()
```

Метод `kill()` спрайта удаляет его из всех групп, в которых он содержится. Есть метод `remove()`, который удаляет только из указанных в качестве аргумента групп. У спрайтов также как у групп есть метод `add()`. Только в данном случае ему передается не объект, а группа.

.

Практические задания.

Создайте столкновение объектов для своей игры, продумайте стратегию победы и проигрыша

Контрольные вопросы:

- 1) Какие использовали модули для работы вашей программы?
- 2) Какие функции используются для работы вашей программы?

Домашнее задание добавьте на выбор пункта меню для своей программы